SLEEP STAGE CLASSIFICATION FROM ECG SIGNALS USING DEEP LEARNING

Under supervision: Dr. Ibrahim Sadek

OUR TEAM

Amany Mahmoud Bakry

Shahd Samir Shaaban

Group 5

Content

Introduction 01

Dataset Overview 02

Data Preparation 03

Data Preprocessing 04

Methodology 05

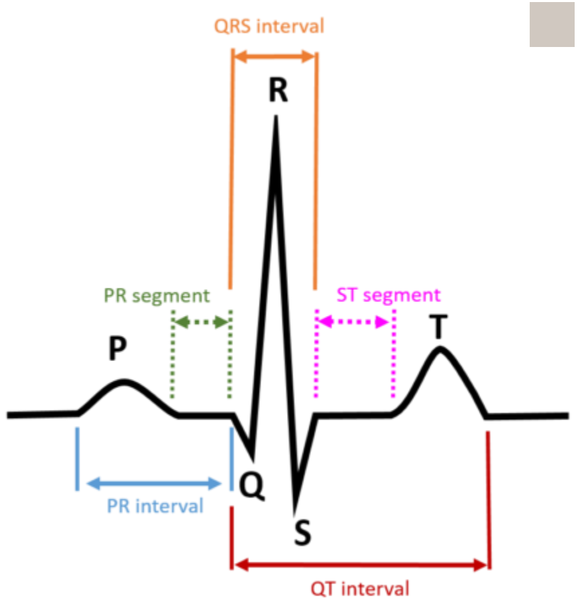
Training 06

Results 07

Introduction

- Sleep stage classification is important for evaluating sleep quality and diagnosing sleep disorders.

- **Traditional sleep staging using Polysomnography (PSG) is:**
  - Time-consuming
  - Subjective
  - Affected by inter-rater variability



- **ECG signals are a promising alternative because they are:**
  - Non-invasive
  - Strongly related to autonomic nervous system activity

- Heart Rate Variability (HRV) provides important physiological information about sleep dynamics.

Introduction

Proposed Framework

- **Hybrid deep learning framework using:**
  - Raw ECG signals
  - HRV features

- **ECG preprocessing steps:**
  - Bandpass filtering
  - Segmentation into 30-second epochs
  - Alignment with sleep stage annotations

- **HRV features extracted from:**
  - Time domain
  - Frequency domain

Introduction

Deep Learning Architecture

- ResNet1D with dilated convolutions extracts local ECG temporal features.
- Transformer Encoder captures global sequence dependencies.
- Dense Neural Network processes HRV features.
- Feature fusion combines ECG and HRV for sleep stage classification.
- **Improvements:** data augmentation, hybrid loss (Focal Loss + Label Smoothing), and AdamW with two-stage training.
- **Goal:** accurate automated ECG-based sleep stage classification.

Dataset Overview

- **Dataset used:** MESA Sleep Dataset

- **Dataset consists of two main folders:**
  - Signals
  - Annotations

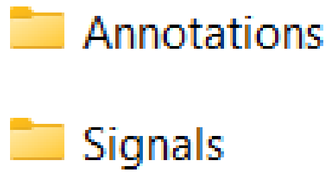
- **Signals folder:**
  - Contains raw physiological recordings
  - Stored in EDF format (.edf)
  - Includes ECG and other physiological signals
  - Total files: 10 EDF files (10 subjects)

- **Annotations folder:**
  - Contains sleep stage labels
  - Total files: 10 NSRR annotation files

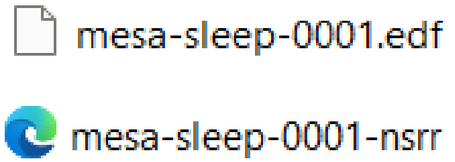
Dataset Overview

Data Structure

- **Each subject has:**
  - One EDF signal file
  - One annotation file



- **Example:**
  - mesa-sleep-0001.edf
  - mesa-sleep-0001-nsrr

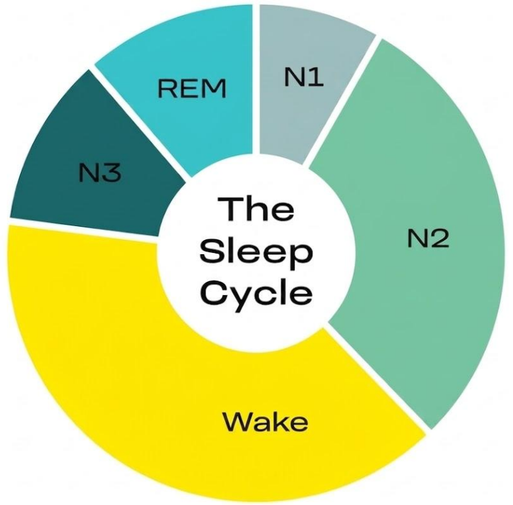


- Recordings span several hours of sleep.
- **Data Type:** Time-series physiological signals
- **Main signal used:** ECG signal
- **File format:** EDF (European Data Format)

Dataset Overview

Sleep Stage Classes

- **Classification based on 5 sleep stages:**
  - Wake
  - N1 (Light Sleep)
  - N2
  - N3 (Deep Sleep)
  - REM



- **Class imbalance exists in the dataset:**
  - N2 is dominant
  - N1 and REM are less frequent

- Data augmentation was applied to handle imbalance.

Dataset Overview

Data Processing & Split

- Continuous ECG recordings were segmented during preprocessing.
- Each subject contains multiple signal segments (epochs).
- **Sampling frequency:**
  - Extracted automatically during preprocessing
  - Consistent within each EDF file
- **Dataset split into:**
  - Training Set
  - Validation Set
  - Test Set
- Subject-level splitting used to avoid data leakage.

Data Preparation

Import Required Libraries

- **NumPy & Pandas:**
  - Numerical computations
  - Data manipulation
- **Matplotlib & Seaborn:**
  - Data visualization
  - Plotting
- **MNE:**
  - Reading EDF physiological signals
- **SciPy:**
  - Butterworth filtering
  - Peak detection
- **Scikit-learn:**
  - Dataset splitting
  - Performance evaluation
  - Class balancing

Data Preparation

Deep Learning Tools

- TensorFlow & Keras used for model development.

- **Main layers used:**
  - Conv1D
  - Dropout
  - Batch Normalization
  - Multi-Head Attention

- **Optimizer:** AdamW

- **Training callbacks:**
  - Learning rate reduction
  - Model checkpointing

- Fixed random seeds applied for reproducibility.

Data Preparation

Data Loading

- ECG recordings and sleep stage annotations were loaded from EDF files.
- MNE library used to read physiological signals.
- Only ECG/EKG channels selected for analysis.
- **Extracted:**
  - Raw ECG signal
  - Sampling frequency
- **Sleep stages mapped into 5 classes:**
  - Wake
  - N1
  - N2
  - N3
  - REM

Data Preparation

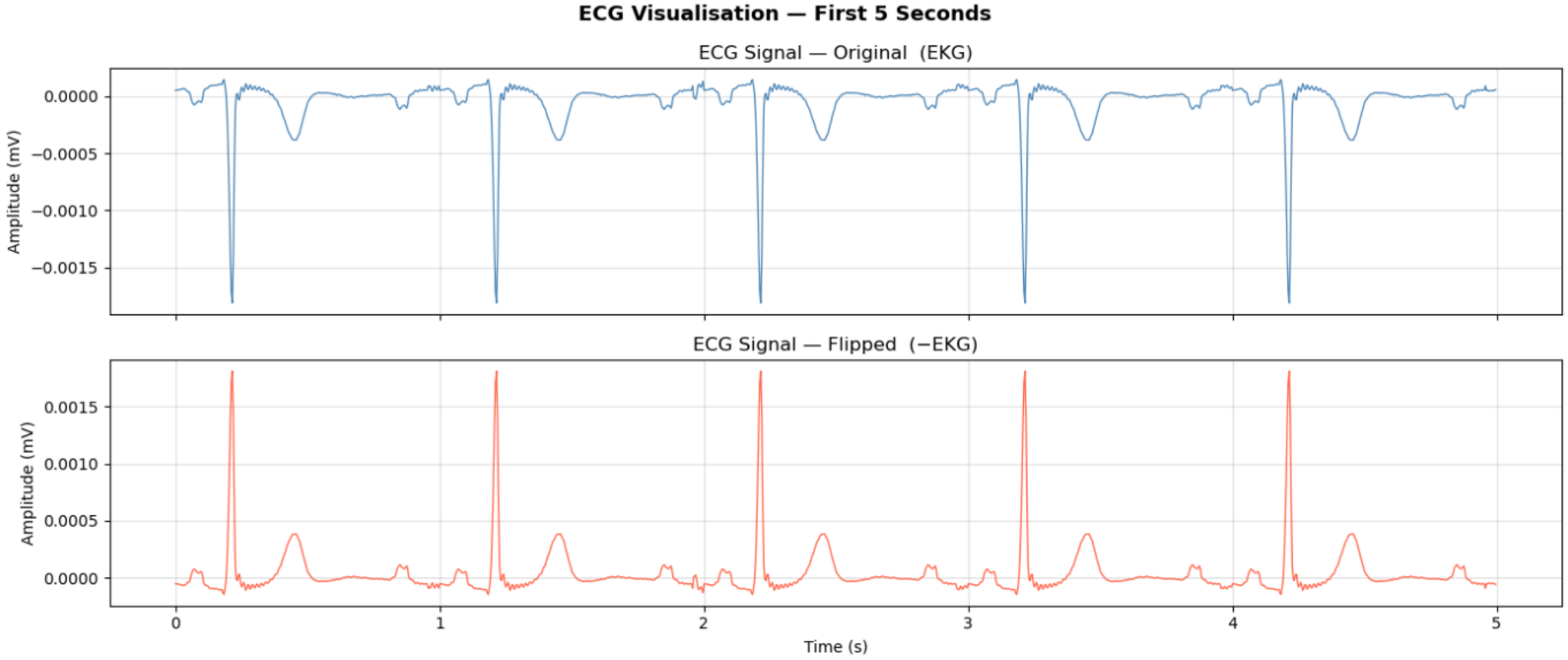
03

Data Loading

Output

✅ Sampling Frequency (Hz): 256.0
✅ Channels: ['EKG', 'EOG-L', 'EOG-R', 'EMG', 'EEG1', 'EEG2', 'EEG3', 'Pres', 'Flow', 'Snore', 'Thor', 'Abdo', 'Leg', 'Therm', 'Pos', 'EKG_Off', 'EOG-L_Off', 'EOG-R_Off', 'EMG_Off', 'EEG1_Off', 'EEG2_Off', 'EEG3_Off', 'Pleth', 'OxStatus', 'SpO2', 'HR', 'DHR']

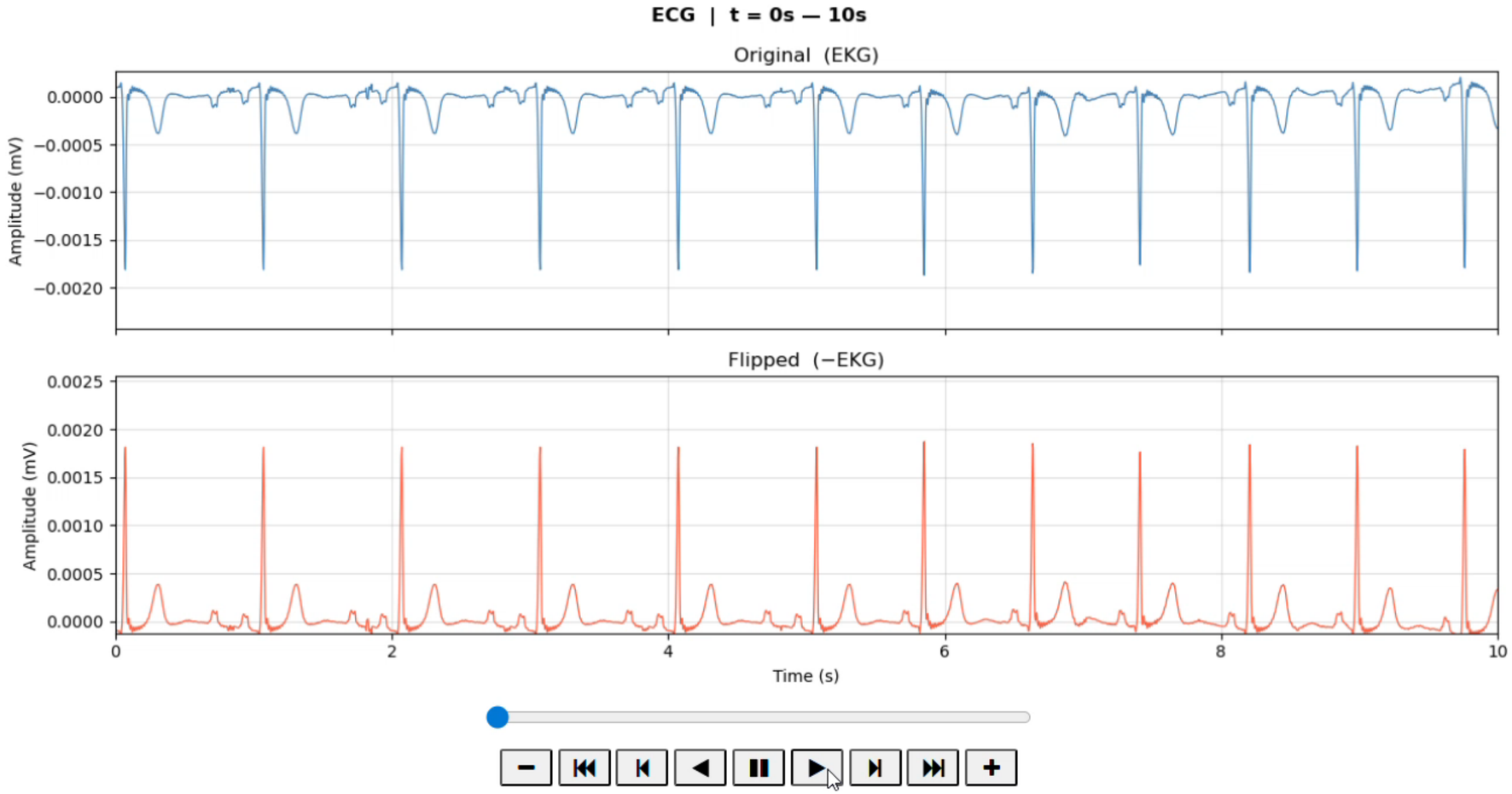
✅ ECG Channels found: ['EKG', 'EKG_Off']



Data Preparation

Data Loading

Output



Data Preparation

Data Loading

Output

```
Found 10 EDF files.
  mesa-sleep-0001 -- 1439 epochs loaded.
  mesa-sleep-0002 -- 1319 epochs loaded.
  mesa-sleep-0006 -- 1079 epochs loaded.
  mesa-sleep-0010 -- 1199 epochs loaded.
  mesa-sleep-0012 -- 1427 epochs loaded.
  mesa-sleep-0014 -- 1679 epochs loaded.
  mesa-sleep-0016 -- 1199 epochs loaded.
  mesa-sleep-0021 -- 1079 epochs loaded.
  mesa-sleep-0027 -- 1199 epochs loaded.
  mesa-sleep-0028 -- 1139 epochs loaded.

Total RAW epochs (before augmentation): 12758
5-class distribution: Counter({0: 5569, 2: 4117, 4: 1261, 1: 942, 3: 869})
```

Data Preparation

Data Organization

- Subjects with missing ECG or annotations were excluded.

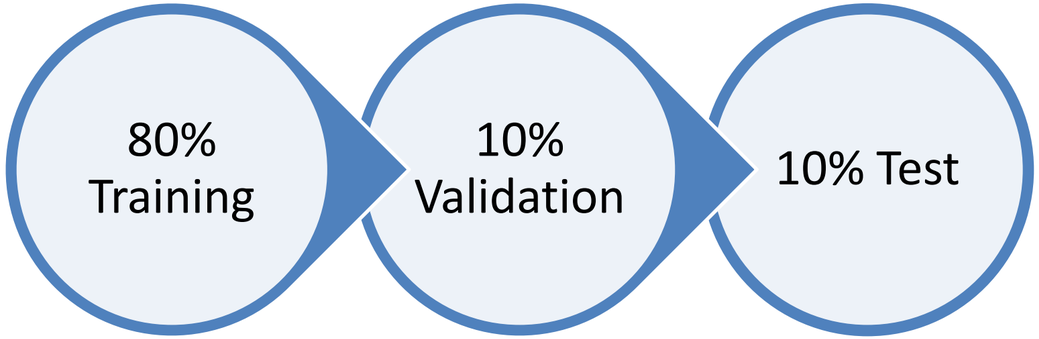
- **Successfully loaded data aggregated into:**
  - ECG signals
  - Labels
  - Sampling frequencies

- **Organized dataset used for:**
  - Preprocessing
  - Feature extraction
  - Model training

Data Preparation

Data Splitting

- Stratified splitting used to preserve class distribution.
- **Dataset divided into:**



- **Splitting process:**
  - 80% allocated for training
  - Remaining 20% divided equally into validation and test sets
- Independent subsets ensure fair model evaluation.

Data Preparation

Data Splitting

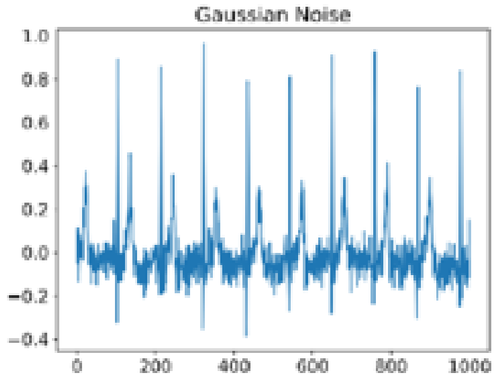
Output

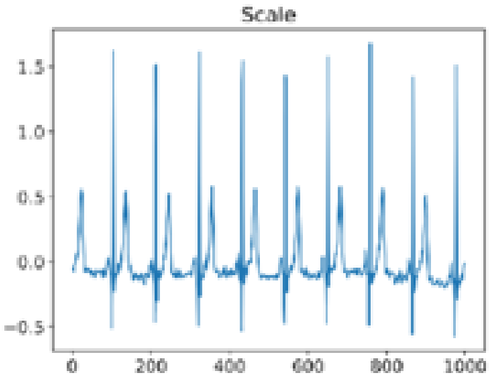
```
Before augmentation:
  Train: (10206, 7680) | Labels: Counter({0: 4455, 2: 3293, 4: 1009, 1: 754, 3: 695})
  Val:   (1276, 7680) | Labels: Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})
  Test:  (1276, 7680) | Labels: Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})
```

Data Preparation

Data Augmentation Strategy

- Applied only to the training set.
- **Goal:**
  - Handle class imbalance
  - Increase minority class samples





- Minority classes augmented until matching majority class size.
- Augmentation applied cyclically on ECG epochs.
- HRV features remained unchanged to preserve physiological consistency.

Data Preparation

Final Balanced Dataset

- Augmented data combined with original training data.
- Final training dataset became balanced across all classes.
- Training data shuffled randomly before training.
- **Validation and test sets were NOT augmented:**
  - Prevents data leakage
  - Ensures fair evaluation

Data Preparation

Final Balanced Dataset

Output

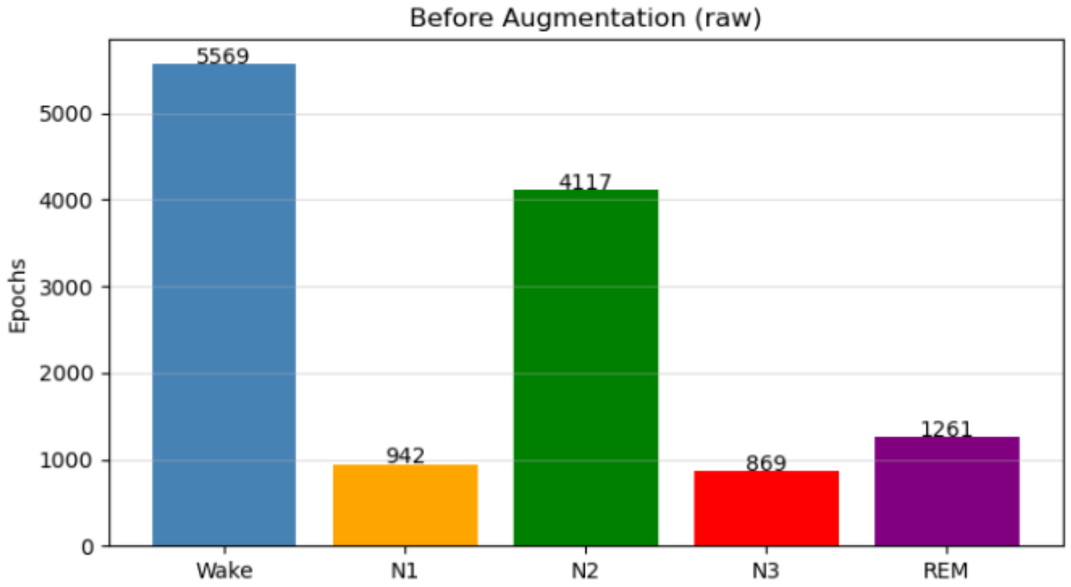
```
Target count per class (= largest class): 4455
  Wake : no augmentation needed (4455 epochs)
  N1 : 754 -> 4455 (adding 3701 augmented)
  N2 : 3293 -> 4455 (adding 1162 augmented)
  N3 : 695 -> 4455 (adding 3760 augmented)
  REM : 1009 -> 4455 (adding 3446 augmented)

After augmentation:
  Train: (22275, 7680) | Labels: Counter({3: 4455, 4: 4455, 2: 4455, 1: 4455, 0: 4455})
  Val:   (1276, 7680)
  Test:  (1276, 7680)
```

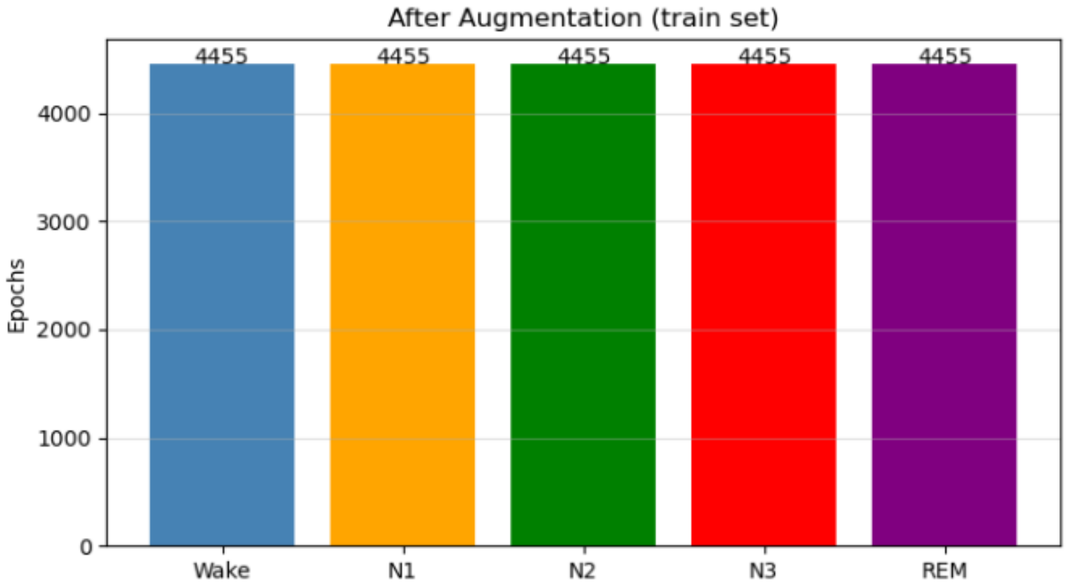
Data Preparation

Final Balanced Dataset

Output

```
Final shapes after normalization:
  X_ecg_tr : (22275, 7680, 1)  X_hrv_tr : (22275, 12)
  X_ecg_val: (1276, 7680, 1)  X_hrv_val: (1276, 12)
  X_ecg_te : (1276, 7680, 1)  X_hrv_te : (1276, 12)
```





Data Preprocessing

Signal Normalization

- **Normalization applied to:**
  - ECG signals
  - HRV features

- **ECG normalization:**
  - Median-based normalization
  - Division by Interquartile Range (IQR)
  - Robust against outliers and artifacts

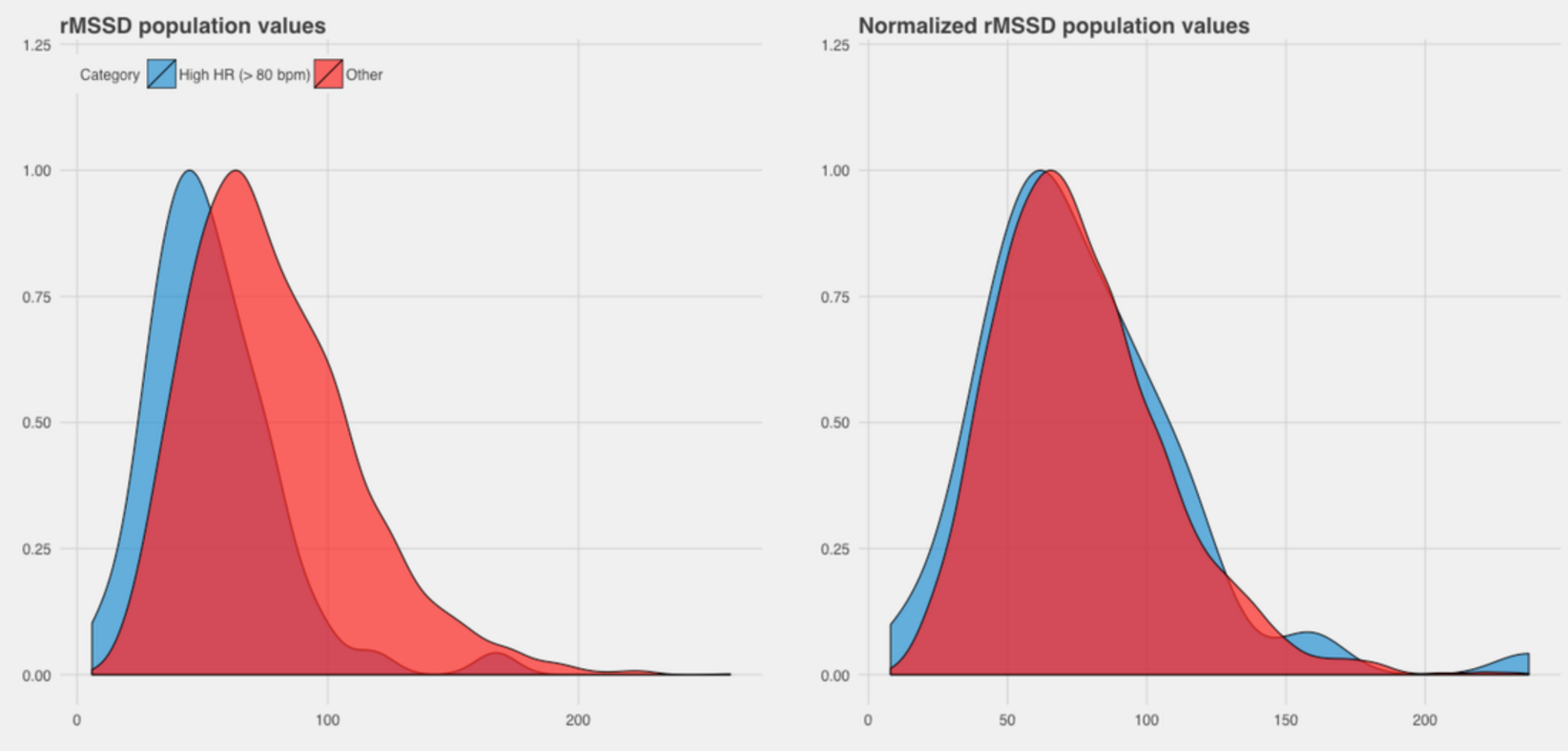
- **HRV normalization:**
  - Mean and standard deviation from training set only
  - Prevents data leakage

- NaN and infinite values replaced for numerical stability.

Data Preprocessing

Signal Normalization

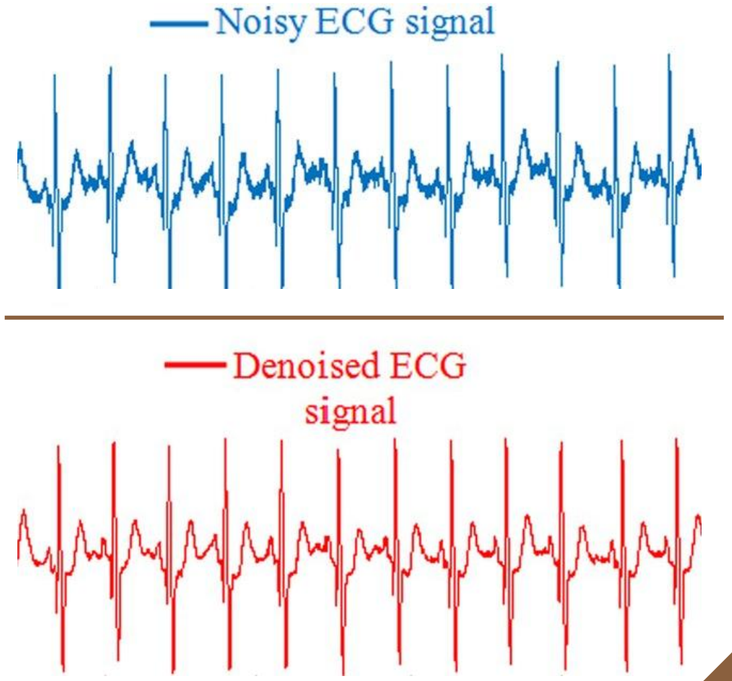
Output



Data Preprocessing

Noise Reduction & Filtering

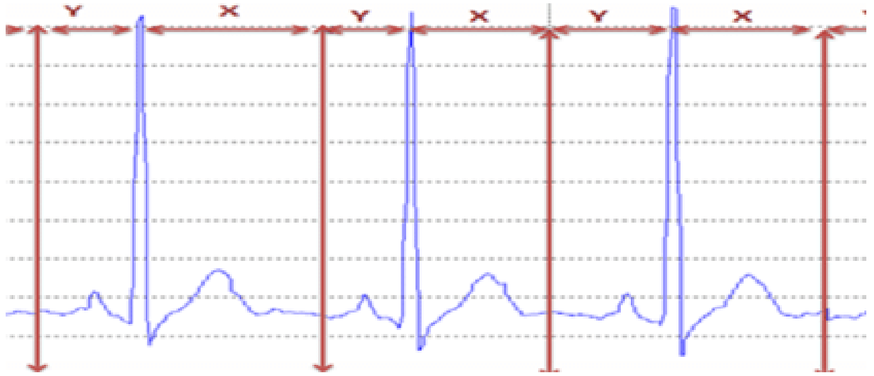
- Bandpass filtering applied to improve ECG signal quality.
- Butterworth Bandpass Filter used.
- **Frequency range:**
  - 0.5 – 40 Hz
- **Purpose:**
  - Remove baseline drift
  - Remove high-frequency noise
  - Preserve important ECG components
- Filtering applied before feature extraction and training.



Data Preprocessing

Signal Segmentation

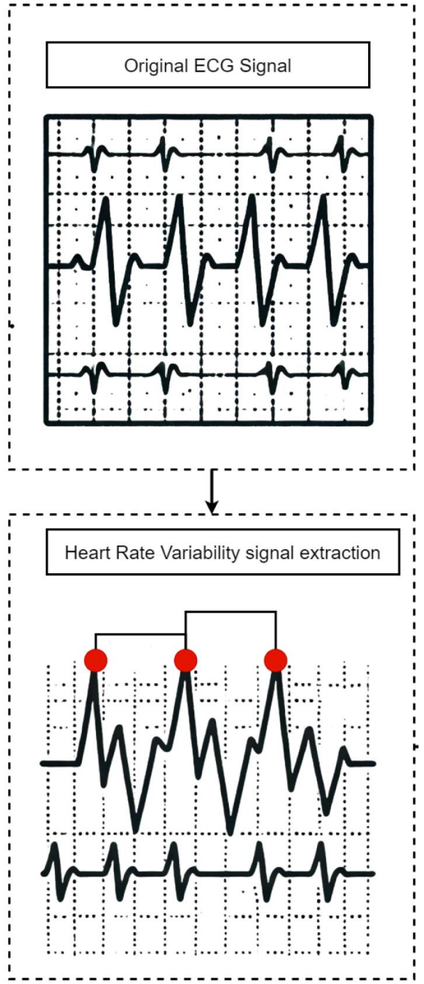
- Continuous ECG signals divided into fixed-length epochs.
- **Epoch length:** 30 seconds
- Non-overlapping segmentation applied.
- Each epoch aligned with its sleep stage annotation.
- Only valid annotated epochs retained for analysis.



Data Preprocessing

HRV Feature Extraction

- HRV features extracted from each ECG epoch.
- R-peaks detected first from ECG signals.
- RR intervals computed for HRV analysis.
- **Extracted features included:**
  - Time-domain features
  - Frequency-domain features
  - Nonlinear features



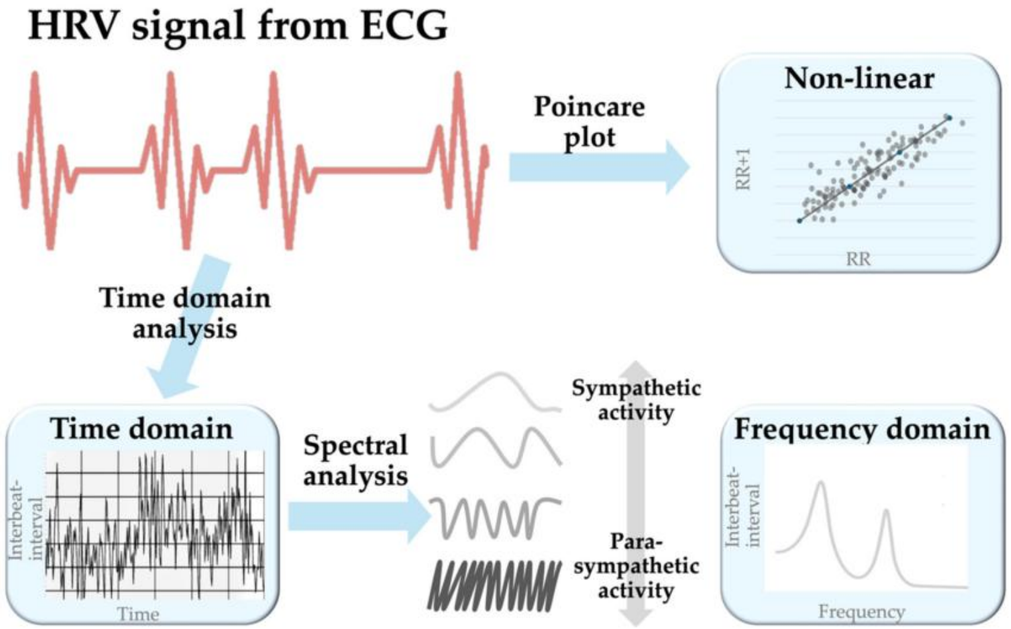
Data Preprocessing

Types of HRV Features

- **Time-domain features:**
  - Mean RR interval
  - Standard deviation
  - RMSSD
  - pNN50
- **Frequency-domain features:**
  - LF Power
  - HF Power
  - LF/HF Ratio
- **Nonlinear features:**
  - SD1
  - SD2
  - SD1/SD2 Ratio
- HRV features enhance sleep stage discrimination.

Data Preprocessing

Types of HRV Features



Data Preprocessing

Class Weights Strategy

- **Class distribution analyzed across:**
  - Training set
  - Validation set
  - Test set

- **After augmentation:**
  - All classes became balanced

- **Uniform class weights assigned:**
  - Each class weight = 1.0

- **Benefits:**
  - Simplifies training
  - Ensures fair class representation

Data Preprocessing

Class Weights Strategy

Output

```
Train labels : Counter({3: 4455, 4: 4455, 2: 4455, 1: 4455, 0: 4455})
Val labels   : Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})
Test labels  : Counter({0: 557, 2: 412, 4: 126, 1: 94, 3: 87})

Class weights (uniform — data already balanced): {0: 1.0, 1: 1.0, 2: 1.0, 3: 1.0, 4: 1.0}
```

Methodology

Deep Learning Architecture

- Hybrid deep learning architecture designed for sleep stage classification.

- **Dual-branch structure:**
  - ECG branch
  - HRV branch

- **ECG branch:**
  - Processes raw ECG signals
  - Uses CNN + ResNet + Transformer

- **HRV branch:**
  - Processes HRV feature vectors
  - Uses Dense layers

- Outputs from both branches fused for final classification.

Methodology

Model Inputs

- **Model accepts two separate inputs:**

1) ECG Input:

- One-dimensional ECG time-series
- Represents 30-second epochs

2) HRV Input:

- Fixed-length HRV feature vector
- Extracted from ECG segments

- **Dual-input design combines:**
  - Raw signal dynamics
  - Physiological statistical features

Methodology

ECG Feature Extraction

- **Initial Conv1D layers:**
  - Extract low-level temporal features

- **ResNet1D with Dilated Convolutions:**
  - Captures long-range dependencies
  - Preserves gradient flow

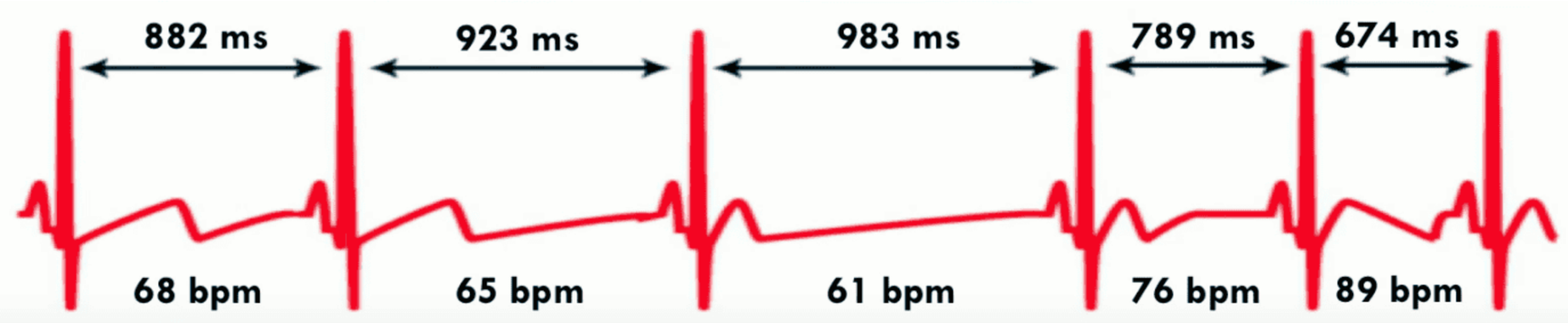
- **Transformer Encoder:**
  - Uses self-attention mechanism
  - Captures global contextual relationships

- **Global Average Pooling:**
  - Generates compact feature representation

Methodology

HRV Branch & Feature Fusion

- HRV branch uses Dense layers to learn higher-level HRV representations.
- ECG and HRV features concatenated together.
- Additional Dense layers applied after fusion.
- **Final output:**
  - Classification into 5 sleep stages



Methodology

Activation Functions

- **ReLU activation used in:**
  - Convolutional layers
  - Residual blocks

- **Benefits of ReLU:**
  - Introduces non-linearity
  - Improves training efficiency

- **GELU activation used in:**
  - Dense layers
  - Transformer layers

- **Softmax activation used at output layer:**
  - Produces probability distribution over sleep stages

Methodology

05

Loss Function Design

- **Hybrid loss function used:**
  - Focal Loss
  - Label Smoothing Cross-Entropy

- **Focal Loss:**
  - Focuses on hard samples
  - Reduces effect of easy samples
  - $\gamma = 3.0$

- **Label Smoothing:**
  - Smoothing factor = 0.1
  - Reduces overconfidence
  - Improves generalization

Output

```
Focal loss defined  (gamma=3.0, label_smooth=0.1)
```

Methodology

Final Loss Function

- Final loss = 50% Focal Loss + 50% Label Smoothing

- **Advantages:**
  - Better handling of class imbalance
  - Improved robustness
  - Better generalization performance

- Designed for multi-class sleep stage classification.

Methodology

Optimizer Configuration

- **Optimizer used:**
  - AdamW

- **Benefits:**
  - Fast convergence
  - Improves generalization
  - Prevents overfitting

- **Configuration:**
  - Learning rate = 1e-3
  - Weight decay = 1e-4

- **Suitable for hybrid architectures:**
  - CNN + Transformer + Dense

Methodology

Model Compilation & Summary

- **Model compiled using:**
  - AdamW optimizer
  - Hybrid loss function

- **Evaluation metric:**
  - Categorical Accuracy

- **Final architecture combines:**
  - CNN - ResNet1D - Transformer Encoder - Dense HRV branch

- Softmax classifier used for final prediction of 5 sleep stages.

Training

Batch Size & Learning Rate

- **Batch size:** 32

- **Benefits:**
  - Stable gradient updates
  - Good computational efficiency

- **Learning rate strategy:**

- **Initial Training:**
  - Learning rate = 1e-3

- **Fine-Tuning:**
  - Learning rate = 2e-5
  - Weight decay = 1e-5

- Reduced learning rate helps prevent overfitting.

Training

Validation Strategy

- **Separate validation set used:**
  - 10% of dataset

- Validation performed after every epoch.

- **Main validation metrics:**
  - Validation Loss
  - Validation Accuracy

- **Purpose:**
  - Monitor generalization performance
  - Evaluate model on unseen data

Training

Training on Balanced Dataset

- Model trained on balanced dataset after augmentation.

- **Equal representation for all sleep stages:**
  - Wake
  - N1
  - N2
  - N3
  - REM

- **Benefits:**
  - Reduces class bias
  - Improves robustness

- Uniform class weights used during training.

Training

Callbacks & Training Control

- **ModelCheckpoint:**
  - Saves best model based on validation accuracy

- **ReduceLROnPlateau:**
  - Reduces learning rate automatically
  - Helps improve convergence

- **Fine-Tuning stage:**
  - Best model from first stage loaded before training

- **Goal:**
  - Improve stability
  - Prevent overfitting

Training

06

Before Fine-Tuning

Output

```
Epoch 1/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 829ms/step - accuracy: 0.6209 - loss: 0.7170
Epoch 1: val_accuracy improved from 0.37147 to 0.61050, saving model to best_sleep_model.keras

Epoch 1: finished saving model to best_sleep_model.keras
697/697 ━━━━━━━━━━━━━━━━━━━━ 588s 843ms/step - accuracy: 0.6331 - loss: 0.7001 - val_accuracy: 0.6105 - val_loss: 0.7126 - learning_rate: 0.0010
Epoch 2/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 822ms/step - accuracy: 0.6535 - loss: 0.6652
Epoch 2: val_accuracy improved from 0.61050 to 0.62931, saving model to best_sleep_model.keras

Epoch 2: finished saving model to best_sleep_model.keras
697/697 ━━━━━━━━━━━━━━━━━━━━ 583s 836ms/step - accuracy: 0.6662 - loss: 0.6526 - val_accuracy: 0.6293 - val_loss: 0.6777 - learning_rate: 0.0010
```

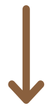
```
Epoch 9/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 810ms/step - accuracy: 0.8425 - loss: 0.4224
Epoch 9: val_accuracy did not improve from 0.68730
697/697 ━━━━━━━━━━━━━━━━━━━━ 574s 824ms/step - accuracy: 0.8511 - loss: 0.4131 - val_accuracy: 0.6787 - val_loss: 0.6825 - learning_rate: 0.0010
Epoch 10/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 807ms/step - accuracy: 0.8624 - loss: 0.4013
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

Epoch 10: val_accuracy improved from 0.68730 to 0.68809, saving model to best_sleep_model.keras

Epoch 10: finished saving model to best_sleep_model.keras
697/697 ━━━━━━━━━━━━━━━━━━━━ 572s 821ms/step - accuracy: 0.8705 - loss: 0.3911 - val_accuracy: 0.6881 - val_loss: 0.6912 - learning_rate: 0.0010

Training complete.
```

Training

After Fine-Tuning

Output

```
Epoch 1/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 818ms/step - accuracy: 0.8857 - loss: 0.3671
Epoch 1: val_accuracy improved from None to 0.76019, saving model to best_sleep_model_finetuned.keras

Epoch 1: finished saving model to best_sleep_model_finetuned.keras
697/697 ━━━━━━━━━━━━━━━━━━━━ 592s 834ms/step - accuracy: 0.9034 - loss: 0.3463 - val_accuracy: 0.7602 - val_loss: 0.5723 - learning_rate: 2.0000e-05
Epoch 2/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 826ms/step - accuracy: 0.9175 - loss: 0.3326
Epoch 2: val_accuracy improved from 0.76019 to 0.76489, saving model to best_sleep_model_finetuned.keras

Epoch 2: finished saving model to best_sleep_model_finetuned.keras
697/697 ━━━━━━━━━━━━━━━━━━━━ 586s 841ms/step - accuracy: 0.9288 - loss: 0.3197 - val_accuracy: 0.7649 - val_loss: 0.5668 - learning_rate: 2.0000e-05
```



```
Epoch 9/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 831ms/step - accuracy: 0.9516 - loss: 0.2863
Epoch 9: val_accuracy did not improve from 0.78370
697/697 ━━━━━━━━━━━━━━━━━━━━ 589s 845ms/step - accuracy: 0.9564 - loss: 0.2783 - val_accuracy: 0.7837 - val_loss: 0.5793 - learning_rate: 8.0000e-06
Epoch 10/10
697/697 ━━━━━━━━━━━━━━━━━━━━ 0s 827ms/step - accuracy: 0.9516 - loss: 0.2847
Epoch 10: val_accuracy did not improve from 0.78370
697/697 ━━━━━━━━━━━━━━━━━━━━ 586s 841ms/step - accuracy: 0.9582 - loss: 0.2770 - val_accuracy: 0.7813 - val_loss: 0.5805 - learning_rate: 8.0000e-06

Fine-tuning complete.
```

Results

Test Set Evaluation

- Final model evaluated on an independent test set.
- **Test data was NOT used during:**
  - Training
  - Validation
- Best model weights loaded from fine-tuning stage.
- **Predictions generated using:**
  - Softmax probabilities
  - Argmax classification

Results

Evaluation Metrics

- **Accuracy:**
  - Overall percentage of correct predictions

- **Balanced Accuracy:**
  - Average recall across all classes
  - Handles class imbalance

- **Cohen’s Kappa:**
  - Measures agreement beyond chance level

- **Classification report generated for:**
  - Wake - N1 - N2 - N3 - REM

Output

```
================================================
  Test Accuracy          : 0.7970  (79.70%)
  Balanced Accuracy      : 0.7339
  Cohen's Kappa          : 0.7093
================================================
```

Results

Precision, Recall & F1-Score

- **Precision:**
  - Correct predicted samples / Total predicted samples

- **Recall:**
  - Correct identified samples / Total actual samples

- **F1-Score:**
  - Harmonic mean of Precision and Recall

- **These metrics provide:**
  - Detailed class-wise performance analysis
  - Better evaluation for imbalanced datasets

Results

Precision, Recall & F1-Score

Output

	precision	recall	f1-score	support
Wake	0.95	0.92	0.93	557
N1	0.40	0.52	0.46	94
N2	0.79	0.71	0.75	412
N3	0.62	0.72	0.67	87
REM	0.69	0.79	0.74	126
accuracy			0.80	1276
macro avg	0.69	0.73	0.71	1276
weighted avg	0.81	0.80	0.80	1276

Results

Training & Validation Curves

- **Model performance analyzed using:**
  - Accuracy curves
  - Loss curves

- **Training curves show:**
  - Learning progress over epochs

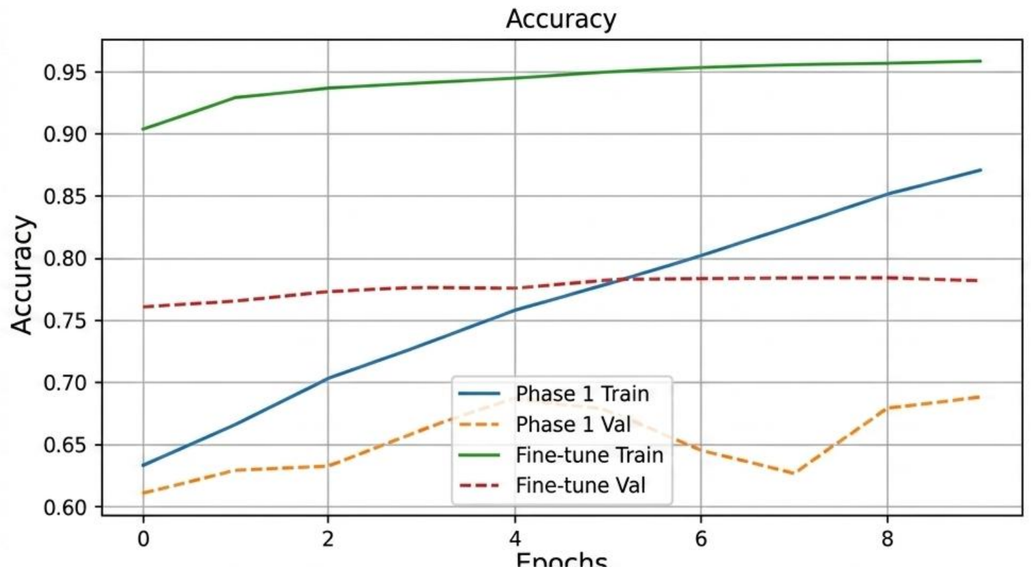
- **Validation curves show:**
  - Generalization performance on unseen data

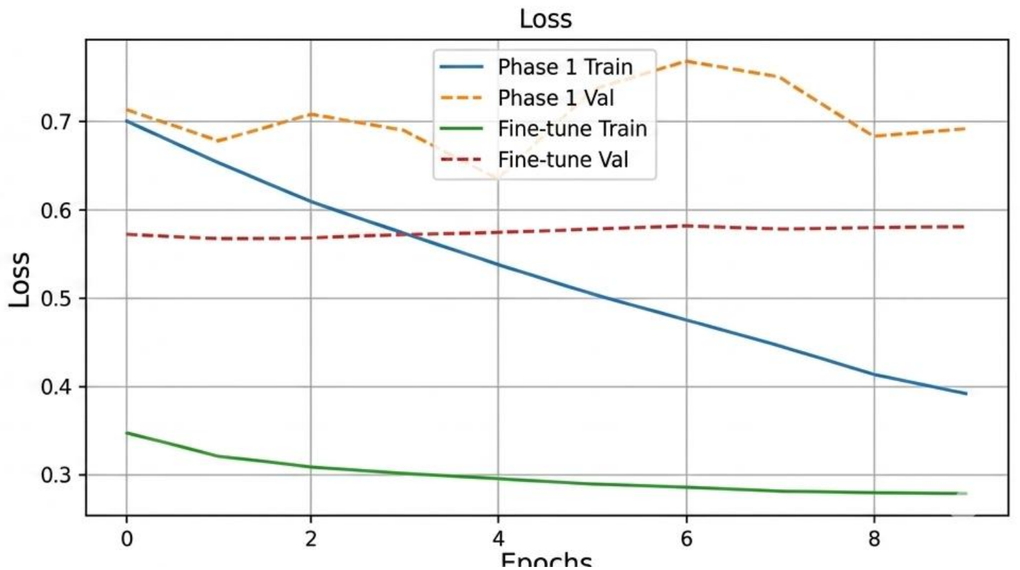
- **Results indicate:**
  - Stable convergence after fine-tuning
  - Improved performance with reduced learning rate

Results

Training & Validation Curves

Output

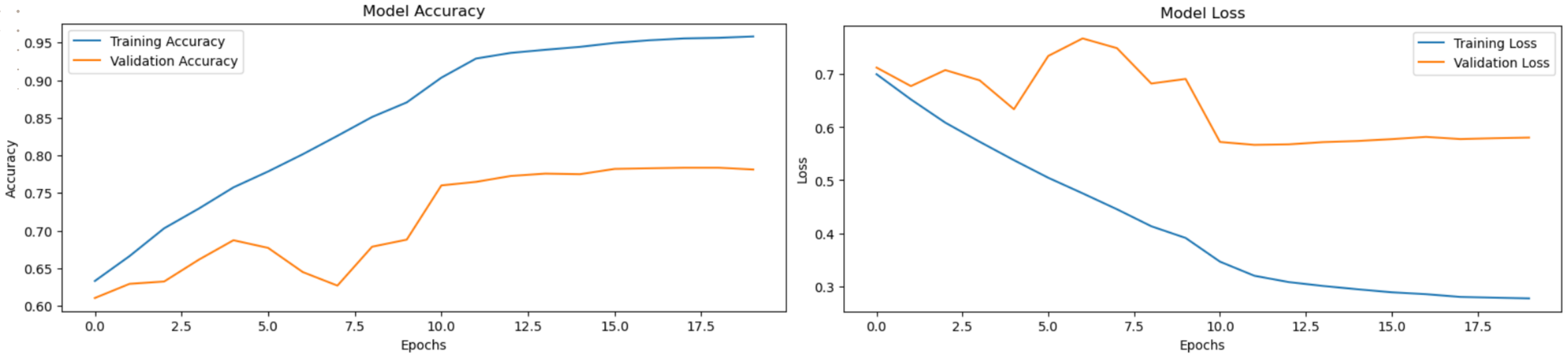




Results

Training & Validation Curves

Output



(Accuracy and Loss Curves After Merging Fine-Tuning with Training)

Results

Class Distribution Analysis

- **Test set contains all sleep stages:**
  - Wake - N1 - N2 - N3 - REM

- Predicted class distribution analyzed.

- **Purpose:**
  - Check model bias
  - Evaluate over/under prediction of classes

- **Result:**
  - Relatively balanced prediction across all classes

Results

Class Distribution Analysis

Output

```
Test set distribution:
  Wake  :   557 samples
  N1    :    94 samples
  N2    :   412 samples
  N3    :    87 samples
  REM   :   126 samples

Prediction distribution:
  Wake  :   539 predicted
  N1    :   121 predicted
  N2    :   371 predicted
  N3    :   101 predicted
  REM   :   144 predicted
```

Results

Confusion Matrix Analysis

- 5×5 Confusion Matrix used for evaluation.

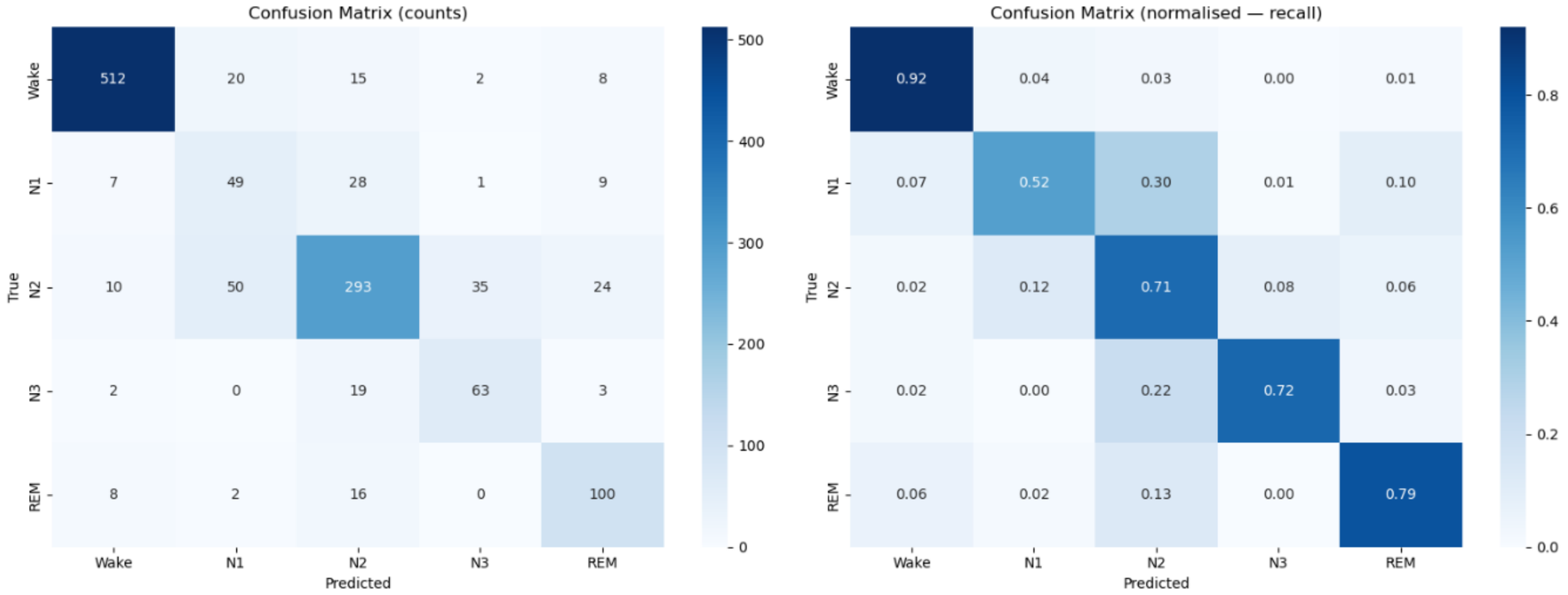
- **Two types analyzed:**
  - Raw Confusion Matrix (absolute values)
  - Normalized Confusion Matrix (recall-based)

- **Shows:**
  - Correct predictions per class
  - Misclassification patterns

Results

Confusion Matrix Analysis

Output



Results

Confusion Patterns

- Model performs well in most sleep stages.

- **Observed confusions between:**
  - N1 ↔ N2
  - N2 ↔ N3

- **Reason:**
  - Physiological similarity between stages

- **Overall:**
  - Strong classification performance across classes

Results

ROC Curve Analysis

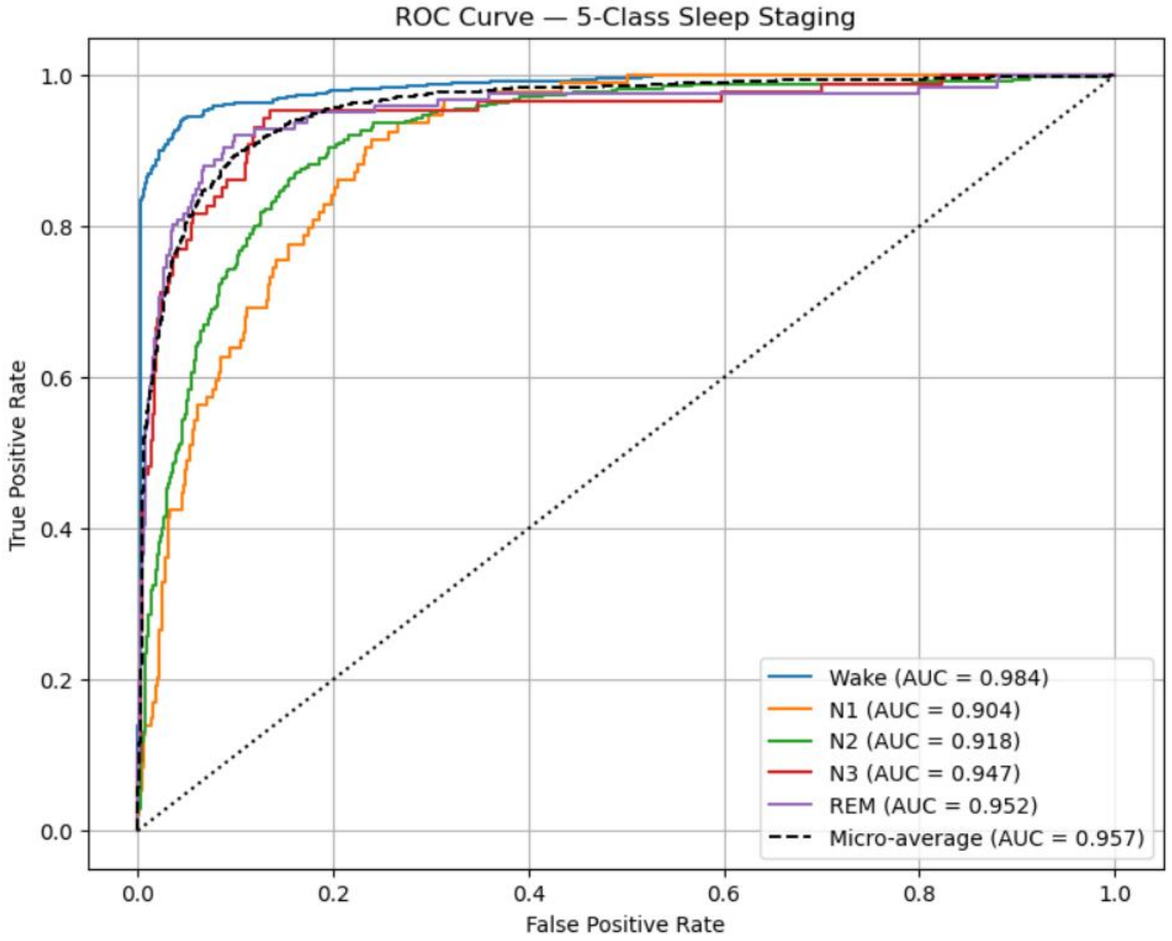
- **ROC (Receiver Operating Characteristic) curves generated using:**
  - One-vs-Rest strategy
  - For all 5 sleep stage classes

- **Each ROC curve shows:**
  - True Positive Rate (TPR)
  - False Positive Rate (FPR)
  - Across different thresholds

Results

ROC Curve Analysis

Output



Results

AUC & Model Performance

- Area Under Curve (AUC) computed for each class.

- **Purpose:**
  - Measure class-wise discriminative ability

- **Micro-average ROC curve used for:**
  - Overall model performance across all classes

- **Results:**
  - High AUC values for all classes

- **Conclusion:**
  - Strong separability between sleep stages
  - Robust classification performance

Results

Discussion: Model Strengths

- Hybrid architecture combines raw ECG and HRV features for better performance.
- ResNet1D captures local and long-range patterns, while the Transformer captures global temporal relationships.
- Data augmentation and class balancing improve generalization and reduce class imbalance bias.
- Focal Loss + Label Smoothing with AdamW provide more stable and effective training.

Results

Discussion: Model Limitations

- **Limited dataset size:**
  - Only 10 subjects
  - May reduce generalization to larger populations

- **HRV dependency:**
  - Relies on accurate R-peak detection
  - Sensitive to noise and artifacts

- **High computational cost:**
  - CNN + Transformer architecture
  - Not ideal for low-power or real-time devices

- **Data augmentation limitation:**
  - Improves robustness but cannot replace large real-world datasets

Discussion

Future Improvements

- Use larger and more diverse datasets to improve generalization.
- Integrate additional signals such as EEG and respiration.
- Develop lightweight models for real-time wearable devices.
- Apply advanced methods like self-supervised and contrastive learning.
- Improve preprocessing and adaptive data augmentation for higher accuracy and reliability.

THANK YOU